

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY Wrong IS
5      PORT (
6          -- Inputs
7          reset : IN STD_LOGIC;
8          clk : IN STD_LOGIC;
9          err_event : IN STD_LOGIC;
10         -- Outputs
11         err_count : OUT STD_LOGIC_VECTOR(1 DOWNTO 0);
12         failed : OUT STD_LOGIC
13     );
14 END Wrong;
15
16 ARCHITECTURE processes OF Wrong IS
17     -- Define the states of the FSM
18     TYPE state IS (idle, err_state0, err_state1, err_state2);
19     SIGNAL present_state, next_state : state;
20 BEGIN
21     -- Process for state registration
22     state_reg : PROCESS (clk)
23     BEGIN
24         IF rising_edge(clk) THEN
25             IF reset = '0' THEN
26                 present_state <= idle;
27             ELSE
28                 present_state <= next_state;
29             END IF;
30         END IF;
31     END PROCESS;
32     nxt_state : PROCESS (present_state, err_event)
33     BEGIN
34         CASE present_state IS
35             -- one case branch required for each state
36             WHEN idle =>
37                 IF err_event = '1' THEN
38                     next_state <= err_state0;
39                 ELSE
40                     next_state <= idle;
41                 END IF;
42             WHEN err_state0 =>
43                 IF err_event = '1' THEN
44                     next_state <= err_state1;
45                 ELSE
46                     next_state <= err_state0;
47                 END IF;
48             WHEN err_state1 =>
49                 IF err_event = '1' THEN
50                     next_state <= err_state2;
51                 ELSE
52                     next_state <= err_state1;
53                 END IF;
54             WHEN err_state2 =>
55                 next_state <= err_state2;
56         END CASE;
57     END PROCESS;
58     outputs : PROCESS (present_state, err_event)
59     BEGIN
60         CASE present_state IS
61             WHEN err_state2 =>
62                 failed <= '1';
63                 err_count <= "11";
64             WHEN err_state1 =>
65                 err_count <= "10";
66                 failed <= '0';
67             WHEN err_state0 =>
68                 err_count <= "01";
69                 failed <= '0';
70             WHEN OTHERS =>
71                 failed <= '0';

```

```
72         err_count <= "00";  
73     END CASE;  
74 END PROCESS;  
75 END processes;
```